

03 — Neural Networks from Scratch

Time: ~4-5 hours | **Level:** Intermediate

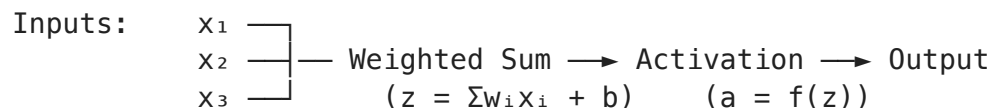
What you'll learn:

- What a neuron actually computes (it's just math)
- Forward pass: input → output
- Loss functions: measuring how wrong the model is
- Backpropagation: the algorithm that makes learning possible
- Gradient descent: how models improve step by step
- Building a multi-layer neural network using only NumPy

Prerequisites: Notebooks 01-02 (NumPy, basic ML concepts)

1. What Is a Neuron?

A biological neuron receives signals, processes them, and fires (or doesn't). An artificial neuron does the same:



Mathematically:

$$z = w_1 x_1 + w_2 x_2 + w_3 x_3 + b$$
$$a = f(z)$$

Where:

- x_i = inputs (features)
- w_i = weights (learned parameters — **these are what the model learns**)
- b = bias (allows the function to shift)
- f = activation function (introduces non-linearity)
- a = output (activation)

Why do we need activation functions?

Without them, stacking layers is pointless: $(Wx + b_1)W_2 + b_2$ is still just a linear function. Activation functions introduce **non-linearity**, allowing the network to learn complex patterns.

```

In [ ]: import numpy as np
import matplotlib.pyplot as plt

# — Common Activation Functions —————
def sigmoid(z):
    """Squashes any value to (0, 1). Used for probabilities."""
    return 1 / (1 + np.exp(-z))

def relu(z):
    """ReLU: max(0, z). The most popular activation in deep learning."""
    return np.maximum(0, z)

def tanh_act(z):
    """Tanh: squashes to (-1, 1). Centered at zero."""
    return np.tanh(z)

# Derivatives (needed for backpropagation)
def sigmoid_derivative(a):
    """d/dz sigmoid(z) = sigmoid(z) * (1 - sigmoid(z))"""
    return a * (1 - a) # a is already sigmoid(z)

def relu_derivative(z):
    """d/dz ReLU(z) = 1 if z > 0, else 0"""
    return (z > 0).astype(float)

# Visualise
z = np.linspace(-5, 5, 200)
fig, axes = plt.subplots(1, 3, figsize=(15, 4))

for ax, func, name, color in [
    (axes[0], sigmoid, 'Sigmoid:  $\sigma(z) = 1/(1+e^{-z})$ ', 'coral'),
    (axes[1], relu, 'ReLU:  $\max(0, z)$ ', 'steelblue'),
    (axes[2], tanh_act, 'Tanh:  $(e^z - e^{-z})/(e^z + e^{-z})$ ', 'seagreen'),
]:
    ax.plot(z, func(z), linewidth=2.5, color=color)
    ax.axhline(0, color='gray', linewidth=0.5)
    ax.axvline(0, color='gray', linewidth=0.5)
    ax.set_title(name)
    ax.grid(True, alpha=0.3)

plt.tight_layout()
plt.show()

print('💡 ReLU is the default for hidden layers in modern deep learning.')
print('    Sigmoid is used for the output layer in binary classification.')
print('    Tanh is used in RNNs and LSTMs (we\'ll see later).')

```

2. Forward Pass — From Input to Prediction

A **forward pass** is computing the output of the network given an input. Let's build a 2-layer neural network by hand.

Input (4 features) → Hidden Layer (8 neurons, ReLU) → Output Layer (1 neuron, Sigmoid)

Step by step:

1. $z_1 = X \cdot W_1 + b_1$ (linear transformation)
2. $a_1 = \text{ReLU}(z_1)$ (activation)
3. $z_2 = a_1 \cdot W_2 + b_2$ (another linear transformation)
4. $\hat{y} = \sigma(z_2)$ (sigmoid → probability)

```
In [ ]: np.random.seed(42)

# — Generate synthetic data —————
# Binary classification: predict high-risk from 4 features
n_samples = 500
X = np.random.randn(n_samples, 4) # 500 samples, 4 features
# True pattern: high-risk if feature[0] + feature[1] > 0.5
y = ((X[:, 0] + X[:, 1] + 0.3 * X[:, 2]) > 0.5).astype(float).reshape(-1, 1)

print(f'Dataset: {X.shape[0]} samples, {X.shape[1]} features')
print(f'Class balance: {y.mean():.2%} positive')

# — Initialise weights (random small values) —————
# Xavier initialisation: scale by 1/sqrt(n_inputs)
n_input, n_hidden, n_output = 4, 8, 1

W1 = np.random.randn(n_input, n_hidden) * np.sqrt(1 / n_input) # (4, 8)
b1 = np.zeros((1, n_hidden)) # (1, 8)
W2 = np.random.randn(n_hidden, n_output) * np.sqrt(1 / n_hidden) # (8, 1)
b2 = np.zeros((1, n_output)) # (1, 1)

print(f'\nNetwork architecture:')
print(f'  Input: {n_input} features')
print(f'  Hidden: {n_hidden} neurons (ReLU) — W1 shape: {W1.shape}')
print(f'  Output: {n_output} neuron (Sigmoid) — W2 shape: {W2.shape}')
print(f'  Total parameters: {W1.size + b1.size + W2.size + b2.size}')
```

```
In [ ]: # — Forward Pass (step by step) —————

def forward(X, W1, b1, W2, b2):
    """
    Forward pass through a 2-layer neural network.

    X: (n_samples, 4) — input features
    W1: (4, 8) — weights for layer 1
    b1: (1, 8) — biases for layer 1
    W2: (8, 1) — weights for layer 2
    b2: (1, 1) — biases for layer 2

    Returns:
        y_hat: predictions (n_samples, 1)
        cache: intermediate values needed for backprop
    """
    # Layer 1: Linear + ReLU
```

```

z1 = X @ W1 + b1      # (500, 4) @ (4, 8) + (1, 8) = (500, 8)
a1 = relu(z1)          # (500, 8) - ReLU activation

# Layer 2: Linear + Sigmoid
z2 = a1 @ W2 + b2      # (500, 8) @ (8, 1) + (1, 1) = (500, 1)
y_hat = sigmoid(z2)    # (500, 1) - probability output

cache = (z1, a1, z2, y_hat) # save for backprop
return y_hat, cache

y_hat, cache = forward(X, W1, b1, W2, b2)
print(f'Predictions shape: {y_hat.shape}')
print(f'First 5 predictions: {y_hat[:5].flatten().round(3)}')
print(f'First 5 actual:      {y[:5].flatten()}')
print(f'\n💡 Predictions are random because weights are random. Training will')

```

3. Loss Functions — Measuring How Wrong We Are

A **loss function** quantifies the error between predictions ($\hat{y}$) and actual values (y).

Binary Cross-Entropy (for classification)

$$L = -\frac{1}{n} \sum_{i=1}^n [y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)]$$

Intuition:

- If actual $y = 1$ and prediction $\hat{y} = 0.99 \rightarrow$ loss is tiny (good!)
- If actual $y = 1$ and prediction $\hat{y} = 0.01 \rightarrow$ loss is huge (bad!)
- It penalises confident wrong predictions heavily

```

In [ ]: def binary_cross_entropy(y_true, y_pred):
        """
        Binary cross-entropy loss.
        Clips predictions to avoid log(0) = -infinity.
        """
        eps = 1e-8 # small number to prevent log(0)
        y_pred = np.clip(y_pred, eps, 1 - eps)
        loss = -np.mean(y_true * np.log(y_pred) + (1 - y_true) * np.log(1 - y_pred))
        return loss

# Calculate loss with random weights
initial_loss = binary_cross_entropy(y, y_hat)
print(f'Initial loss (random weights): {initial_loss:.4f}')
print(f'Expected initial loss for balanced data: ~{-np.log(0.5):.4f} (= -log 0.5)')
print(f'Perfect loss would be: 0.0')

# Show how loss changes with different predictions
predictions = np.linspace(0.01, 0.99, 100)
loss_when_y1 = -np.log(predictions) # loss when actual = 1
loss_when_y0 = -np.log(1 - predictions) # loss when actual = 0

```

```
plt.figure(figsize=(8, 5))
plt.plot(predictions, loss_when_y1, 'b-', linewidth=2, label='Actual = 1 (sh
plt.plot(predictions, loss_when_y0, 'r-', linewidth=2, label='Actual = 0 (sh
plt.xlabel('Prediction ( $\hat{y}$ )')
plt.ylabel('Loss')
plt.title('Binary Cross-Entropy: Penalises confident wrong predictions')
plt.legend()
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
```

4. Backpropagation — The Heart of Learning

Backpropagation answers: "How much should I adjust each weight to reduce the loss?"

It uses the **chain rule** from calculus to compute gradients (derivatives) of the loss with respect to every weight in the network.

The Chain Rule

If L depends on $\hat{y}$, which depends on z_2 , which depends on W_2 :

$$\frac{\partial L}{\partial W_2} = \frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial z_2} \cdot \frac{\partial z_2}{\partial W_2}$$

We compute gradients **backwards** through the network (output $\rightarrow$ hidden $\rightarrow$ input), hence "back-propagation".

Forward: $X \rightarrow z_1 \rightarrow a_1 \rightarrow z_2 \rightarrow \hat{y} \rightarrow \text{Loss}$
 Backward: $X \leftarrow dz_1 \leftarrow da_1 \leftarrow dz_2 \leftarrow d\hat{y} \leftarrow d\text{Loss}$

```
In [ ]: def backward(X, y, W1, b1, W2, b2, cache):
        """
        Backpropagation: compute gradients of loss w.r.t. all weights.

        This is the MOST IMPORTANT function in deep learning.
        Every framework (PyTorch, TensorFlow) automates this,
        but understanding it is crucial.
        """
        z1, a1, z2, y_hat = cache
        n = X.shape[0]

        # — Step 1: Gradient of loss w.r.t. output —————
        # dL/d $\hat{y}$  for BCE = ( $\hat{y} - y$ ) / ( $\hat{y}(1-\hat{y})$ )
        #  $d\hat{y}/dz_2$  for sigmoid =  $\hat{y}(1-\hat{y})$ 
        # Combined:  $dL/dz_2 = \hat{y} - y$ 
        dz2 = y_hat - y  # (500, 1) – this elegant result is s

        # — Step 2: Gradients for W2 and b2 —————
        #  $z_2 = a_1 @ W_2 + b_2$ 
```

```

# dL/dW2 = a1T @ dz2
dW2 = (a1.T @ dz2) / n # (8, 1)
db2 = np.mean(dz2, axis=0, keepdims=True) # (1, 1)

# — Step 3: Propagate gradient back to hidden layer —
# dL/da1 = dz2 @ W2T
da1 = dz2 @ W2.T # (500, 8)

# — Step 4: Through ReLU activation —
# dL/dz1 = dL/da1 * ReLU'(z1)
dz1 = da1 * relu_derivative(z1) # (500, 8) – zero gradient where z1 < 0

# — Step 5: Gradients for W1 and b1 —
dW1 = (X.T @ dz1) / n # (4, 8)
db1 = np.mean(dz1, axis=0, keepdims=True) # (1, 8)

return dW1, db1, dW2, db2

# Test backpropagation
dW1, db1, dW2, db2 = backward(X, y, W1, b1, W2, b2, cache)
print('Gradient shapes (should match weight shapes):')
print(f' dW1: {dW1.shape} (W1: {W1.shape})')
print(f' db1: {db1.shape} (b1: {b1.shape})')
print(f' dW2: {dW2.shape} (W2: {W2.shape})')
print(f' db2: {db2.shape} (b2: {b2.shape})')

```

5. Gradient Descent — The Learning Algorithm

Now that we have gradients (direction of steepest increase in loss), we **update weights in the opposite direction** to reduce loss:

$$W_{\text{new}} = W_{\text{old}} - \alpha \cdot \frac{\partial L}{\partial W}$$

Where α is the **learning rate** — how big a step we take.

- Too small: training is very slow
- Too large: training oscillates and diverges
- Just right: smooth convergence to low loss

```

In [ ]: # — Full Training Loop —
def train_network(X, y, n_hidden=8, learning_rate=0.1, epochs=1000, print_ev
    """Train a 2-layer neural network from scratch."""
    np.random.seed(42)
    n_input = X.shape[1]
    n_output = 1

    # Initialise weights
    W1 = np.random.randn(n_input, n_hidden) * np.sqrt(2 / n_input) # He ini
    b1 = np.zeros((1, n_hidden))
    W2 = np.random.randn(n_hidden, n_output) * np.sqrt(2 / n_hidden)
    b2 = np.zeros((1, n_output))

```

```

losses = []

for epoch in range(epochs):
    # Forward pass
    y_hat, cache = forward(X, W1, b1, W2, b2)
    loss = binary_cross_entropy(y, y_hat)
    losses.append(loss)

    # Backward pass
    dW1, db1, dW2, db2 = backward(X, y, W1, b1, W2, b2, cache)

    # Update weights (gradient descent)
    W1 -= learning_rate * dW1
    b1 -= learning_rate * db1
    W2 -= learning_rate * dW2
    b2 -= learning_rate * db2

    if epoch % print_every == 0:
        acc = ((y_hat > 0.5) == y).mean()
        print(f'Epoch {epoch:4d} | Loss: {loss:.4f} | Accuracy: {acc:.3f}')

return W1, b1, W2, b2, losses

# Train!
W1, b1, W2, b2, losses = train_network(X, y, n_hidden=8, learning_rate=0.5,

```

```

In [ ]: # — Visualise the learning process —————
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# Loss curve
axes[0].plot(losses, linewidth=1.5, color='coral')
axes[0].set_xlabel('Epoch')
axes[0].set_ylabel('Loss (Binary Cross-Entropy)')
axes[0].set_title('Training Loss Over Time')
axes[0].grid(True, alpha=0.3)

# Compare learning rates
for lr, color in [(0.01, 'blue'), (0.1, 'orange'), (0.5, 'green'), (2.0, 'red')]:
    _, _, _, _ = train_network(X, y, learning_rate=lr, epochs=500, print_every=10)
    axes[1].plot(l, label=f'lr={lr}', linewidth=1.5, color=color)

axes[1].set_xlabel('Epoch')
axes[1].set_ylabel('Loss')
axes[1].set_title('Effect of Learning Rate')
axes[1].legend()
axes[1].grid(True, alpha=0.3)

plt.tight_layout()
plt.show()

print('💡 Too small (0.01): slow convergence')
print('    Just right (0.1–0.5): smooth descent')
print('    Too large (2.0): may oscillate or diverge')

```

6. Putting It All Together

Let's verify our from-scratch network actually works on new data:

```
In [ ]: # — Evaluate on NEW data (test set) —————
X_test = np.random.randn(200, 4)
y_test = ((X_test[:, 0] + X_test[:, 1] + 0.3 * X_test[:, 2]) > 0.5).astype(float)

y_pred, _ = forward(X_test, W1, b1, W2, b2)
acc = ((y_pred > 0.5) == y_test).mean()

print(f'Test Accuracy: {acc:.3f}')
print(f'Test Loss: {binary_cross_entropy(y_test, y_pred):.4f}')
print(f'\n💡 We built a neural network from SCRATCH that can generalise to new data!')
print('    Next: PyTorch automates all of this (backprop, gradients, GPU acceleration)')
```

🔑 Summary: What a Neural Network Does

1. FORWARD PASS: Input $\rightarrow$ (Weights $\times$ Input + Bias) $\rightarrow$ Activation $\rightarrow$... $\rightarrow$ Prediction
2. LOSS: Compare prediction to ground truth $\rightarrow$ single number (how wrong)
3. BACKWARD PASS: Chain rule $\rightarrow$ gradient of loss w.r.t. every weight
4. UPDATE: Weight $\text{--}=\text{learning_rate} \times \text{gradient}$
5. REPEAT steps 1–4 for many epochs until loss is low

That's it. Every neural network — from a 2-layer net to GPT-4 — follows this exact loop. The differences are in:

- Architecture (how layers are connected)
- Scale (billions of parameters vs hundreds)
- Optimisation tricks (Adam, learning rate schedules)
- Data engineering

✅ Key Takeaways

1. A **neuron** = linear transformation + activation function
2. **Forward pass** = computing output from input
3. **Loss function** = measuring prediction error
4. **Backpropagation** = chain rule to compute gradients
5. **Gradient descent** = update weights opposite to gradient direction
6. **Learning rate** is the most important hyperparameter

Next: [04 — PyTorch & Deep Learning](#) — automate everything above

